

Práctica: CSP y Búsqueda Heurística

Heurística y Optimización



Universidad
Carlos III de Madrid

Grupo 81:

Estudiante:	Montserrat Martis Contreras	NIA:	100522371
Estudiante:	José Barrio Bielsa	NIA:	100522205

Introducción

El objetivo principal de esta práctica ha sido modelar y resolver problemas mediante técnicas de satisfacción de restricciones y algoritmos de búsqueda heurística, implementando soluciones eficientes y analizando su comportamiento frente a instancias reales. La práctica se divide en dos problemas: modelización y resolución del puzzle BINAIRO como un problema de satisfacción de restricciones (CSP) y la búsqueda del camino más corto en grafos reales usando algoritmos informados y no informados.

En la primera parte se propone un modelo formal que representa cada casilla como una variable binaria y que incorpora restricciones globales de conteo y restricciones locales que prohíben tres fichas del mismo color consecutivas. La implementación se realizó con la librería python-constraint, aplicando técnicas de propagación y heurísticas de selección de variables para mejorar la eficiencia. En la segunda parte se diseñó e implementó una estructura de grafo eficiente y se compararon algoritmos de búsqueda exhaustiva con versiones heurísticas óptimas, empleando una heurística geométrica basada en coordenadas geográficas para guiar la búsqueda.

El informe presenta, además de la descripción técnica de los modelos y algoritmos, los criterios de diseño adoptados, los detalles de implementación relevantes y un análisis empírico que compara tiempos de ejecución y escalabilidad frente a instancias de distinto tamaño.

Parte 1: CSP

- Modelización del problema

En esta parte debemos modelar el puzzle BINAIRO en el que, dado un tablero de $n \times n$ casillas, se debe rellenar el tablero con piezas blancas o negras de modo que se cumplan las restricciones: igual número de blancos y negros en cada fila y en cada columna, y no permitir más de dos discos del mismo color consecutivos en ninguna fila ni columna. Las casillas pueden venir preasignadas en la entrada.

Variables y Dominio:

Para cada casilla (r,c) en un tablero $n \times n$ definimos una variable booleana X_{rc} :

- $X_{rc} = 1$ si la casilla $(r;c)$ contiene un disco **negro**.
- $X_{rc} = 0$ si la casilla $(r;c)$ contiene un disco **blanco**.

donde r es el número de fila y c es el número columna tal que $r, c \in \{1, \dots, n\}$

Por lo que, una casilla que no tiene un valor preasignado tiene un dominio $D_{rc} = \{0, 1\}$.

Si el archivo de entrada asigna un valor a una casilla, su dominio se reduce a ese valor (asignación inicial).

Para cada problema se definirán n^2 variables.

Restricciones:

1. Restricción de asignación inicial.

Si en el archivo de entrada se asigna un valor a una casilla se añade la aserción correspondiente:

- Si la casilla $(r;c)$ está fijada a negro: X_{rc} .
- Si la casilla $(r;c)$ está fijada a blanco: $\neg X_{rc}$.

2. Restricción de conteo.

El número de piezas blancas y negras en cada fila, y en cada columna, debe ser el mismo.

Para cada fila r ,

$$\sum_{j=1}^n X_{rj} = \frac{n}{2}$$

y para cada columna c ,

$$\sum_{i=1}^n X_{ic} = \frac{n}{2}$$

Nota: estas son restricciones aritméticas; requieren que n sea par para que exista solución.

3. Restricción de adyacencia.

No es válido colocar más de dos piezas del mismo color consecutivamente en una fila o columna. Para cada fila r y cada posición de columna j con $1 \leq j \leq n - 2$, se prohíben tres negros y tres blancos en la ventana $(j, j+1, j+2)$:

- Prohibir tres negros consecutivos

$$\neg (X_{r,j} \wedge X_{r,j+1} \wedge X_{r,j+2}) = (\neg X_{r,j} \vee \neg X_{r,j+1} \vee \neg X_{r,j+2})$$

- Prohibir tres blancos consecutivos:

$$\neg (\neg X_{r,j} \wedge \neg X_{r,j+1} \wedge \neg X_{r,j+2}) = (X_{r,j} \vee X_{r,j+1} \vee X_{r,j+2})$$

También se añaden las mismas dos restricciones para cada columna c y cada posición de fila i con $1 \leq i \leq n-2$.

- Implementación del modelo

La implementación modela BINAIRO como un problema de satisfacción de restricciones (CSP) y se ha desarrollado usando la librería python-constraint. Las restricciones del enunciado se implementan de forma directa y explícita:

- **Restricción de asignación inicial:**

Las fijaciones iniciales del fichero de entrada se incorporan reduciendo el dominio de la variable correspondiente mediante **InSetConstraint**, una restricción que ya viene dada por python-constraint.

- **Restricción de conteo:**

Para cada fila y cada columna se aplica **ExactSumConstraint**, otra restricción dada por python-constraint, de modo que la suma de los valores (número de negros) sea exactamente la mitad de n . Esta elección aprovecha la codificación binaria y permite expresar la igualdad de conteos como una única restricción global por fila/columna.

- **Restricción de adyacencia:**

Se añade una restricción sobre cada ventana de tamaño 3 en filas y columnas mediante una función booleana **no_three_in_row(a,b,c)** que devuelve **not (a == b == c)**, lo que solo resultará en True si A no es igual a B o si B no es igual a C.

Las decisiones de diseño se justifican por criterios de claridad y eficiencia: la representación binaria simplifica las restricciones aritméticas, el uso de restricciones globales de suma reduce el número de restricciones necesarias y las restricciones de adyacencia tienen un coste de evaluación bajo.

Además, separamos las responsabilidades en módulos auxiliares (**parser.py** para lectura de los archivos de entrada, **board.py** y **pieces.py** para gestionar la representación del tablero) mantiene el código organizado y facilita pruebas y depuración.

- Análisis de Resultados

Complejidad del problema.

La asignación inicial genera tantas restricciones como casillas preasignadas, si hay k casillas preasignadas tendremos k restricciones. De la restricción de conteo existirán n para cada fila y n para cada columna, es decir $2n$ restricciones. De la restricción de adyacencia existirá una por cada ventana de tres, por cada fila hay $n - 2$ ventanas de tamaño 3, luego $n(n - 2)$ restricciones por fila y por columna. En total, $2n(n - 2)$ restricciones. Sumando todos los términos anteriores, el total de restricciones sería $2n + 2n(n - 2) + k$, que simplificado sería $2n^2 - 2n + k$.

Casos de Prueba.

Para analizar esta primera parte realizamos pruebas con tres tamaños de tablero distintos variando la cantidad de casillas preasignadas, k . Vamos a ir variando el parámetro k según distintos porcentajes del total de casillas: 0%, 10%, 30% y 50%.

- Caso 1: Tamaño 4x4

En este caso el número de casillas, y de variables, es 16, por lo que escalando el parámetro k obtenemos los siguientes resultados. Repetimos cada caso tres veces y obtenemos la media del tiempo.

k	restricciones ($2n^2 - 2n + k$)	input utilizado	soluciones encontradas	tiempo medio (segundos)
0 (0%)	24		90	0.0129
2 (10%)	26	X...O..	20	0.0046
		X... O...	30	0.0070

		X... X...	15	0.0057
5 (30%)	29	X..O ..X. ..O.. ..O.	5	0.0023
8 (50%)	32	X..O O.X. ..O.X ..OX	2	0.0024

Utilizando $k = 2$, es interesante destacar cómo la posición y el valor de las casillas preasignadas puede agrandar o reducir el número de soluciones encontradas y, por tanto, el tiempo de resolución.

- Caso 2: Tamaño 6x6

k	restricciones ($2n^2 - 2n + k$)	input utilizado	soluciones encontradas	tiempo medio (segundos)
0 (0%)	60		11222	3.9220
4 (10%)	64	X....O. ..O.. X.....	787	0.4459
11 (30%)	71	O...X .O.... .X..O. ..XO.. ..O.X.. X....O	59	0.0785
18 (50%)	78	O..X.X .OX.XO OX..O. ..XO.X ..O.XO. X....O	1	0.0075

- Caso 3: Tamaño 8x8

k	restricciones ($2n^2 - 2n + k$)	input utilizado	soluciones encontradas	tiempo medio (segundos)
0 (0%)	112		—	Tarda más de 20 minutos.
6 (10%)	118	X.....O. ...O....X.....O..X	188913	646.3417
19 (30%)	131	X.O..O.. ...X.O.. X..O...O ..X...X .X..O.. ...O..X. X...O.. .O.X...X	42	0.2407
32 (50%)	144	X..X.O.O OX...X.X X..O.X.O O.X.O..X X..XOX.O .O.O.OX. X.O..O.. O..X.XO.	4	0.0164

En general, podemos observar como el tiempo de ejecución crece rápidamente porque el problema tiene un espacio de búsqueda exponencial en el número de variables n^2 . En la implementación actual se añaden $2n^2 - 2n + k$ restricciones, y la búsqueda explora combinaciones binarias sujetas a esas restricciones. Esto resulta en que a mayor número de restricciones, menos posibles soluciones y, por tanto, menor tiempo de resolución.

Parte 2: Algoritmos de búsqueda

- Modelización del problema

Este problema de encontrar la ruta más corta entre dos puntos en un mapa se puede modelar como un problema de búsqueda del camino de coste mínimo en un grafo.

Representación del Grafo

El mapa de carreteras se representa como un grafo dirigido y ponderado, $G = (V, E, w)$, donde:

- Vértices (V): Cada vértice del grafo corresponde a una ubicación geográfica específica en el mapa. Cada vértice está identificado por un ID numérico único y tiene asociadas sus coordenadas geográficas (latitud y longitud), que se extraen del fichero ‘.co’.
- Arcos (E): Cada arco dirigido en el grafo representa un tramo de carretera que conecta dos vértices en una dirección específica. La existencia de un arco de un vértice u a un vértice v significa que es posible viajar directamente desde u hasta v . Cada arco tiene un coste asociado, w , que es la distancia en metros de ese tramo de carretera, obtenida del fichero ‘.gr’.

Esta estructura se almacena en una clase **Grafo**, diseñada para construir una representación eficiente en memoria, como una lista de adyacencia. Esta permite consultar rápidamente a los vecinos de cualquier vértice y el coste de los arcos que los conectan.

Espacio de Estados

Para la búsqueda de la solución, se define un espacio de estados donde cada estado representa un punto en el proceso de búsqueda. Un estado se caracteriza por el vértice actual en el que nos encontramos. Para implementar algoritmos de búsqueda informada como A^* , cada nodo de búsqueda que generemos contendrá la siguiente información:

- ID del vértice al que corresponde.
- Coste acumulado (g): La distancia real recorrida desde el vértice de origen hasta el vértice actual.
- Valor heurístico (h): Una estimación del coste restante desde el vértice actual hasta el vértice de destino.
- Coste total estimado (f): La suma ‘ $f = g + h$ ’ se utiliza para priorizar la expansión de nodos.

- **Nodo padre:** Una referencia al estado anterior en el camino, lo que permite reconstruir la ruta óptima una vez que se alcanza el destino.

El estado inicial corresponde al nodo de búsqueda que representa el origen. Para este estado, el coste acumulado g es 0. El estado final se alcanza cuando el nodo a expandir corresponde al destino.

Operadores de Transición

Los operadores de transición entre estados consisten en moverse desde un vértice actual u a uno de sus vértices adyacentes v . El coste de aplicar este operador es el peso del arco que conecta u y v , el cual se suma al coste acumulado ' g ' para llegar a u .

Heurística

Para guiar la búsqueda de manera eficiente y reducir el número de nodos expandidos en comparación con un algoritmo de fuerza bruta (como Dijkstra o búsqueda de coste uniforme), se utiliza una función heurística $h(n)$. Esta función estima la distancia que queda desde un nodo n hasta el destino.

Una heurística admisible y consistente para este problema es la distancia Haversine, que estima la distancia de gran círculo entre dos coordenadas y por ello no sobreestima el coste mínimo real de desplazamiento sobre la superficie terrestre, cumpliendo la condición de admisibilidad necesaria para la optimalidad de A^* . Además, al ser una métrica que satisface la desigualdad triangular lo que implica que para cualquier arista (n, n') se cumple $h(n) \leq c(n, n') + h(n')$, por tanto la heurística es consistente (monótona) y garantiza que los valores ' $f = g + h$ ' no decrezcan al expandir nodos. Esta heurística nunca sobreestimaré el coste real, garantizando que algoritmos como A^* encuentren la solución óptima. El cálculo se realiza utilizando las coordenadas geográficas de los vértices almacenadas en la estructura del grafo.

- **Implementación del algoritmo**

Para resolver el problema del camino más corto, se han implementado dos algoritmos de búsqueda clásicos que nos permitirán comparar la eficiencia de una búsqueda informada frente a una de fuerza bruta. El algoritmo principal y más eficiente es A^* , mientras que el algoritmo de Dijkstra se utiliza como base de comparación (fuerza bruta). La implementación sigue una estructura genérica de búsqueda en grafos, utilizando dos estructuras de datos fundamentales:

- Lista Abierta (Open): almacena los nodos que han sido generados pero aún no han sido expandidos. Para garantizar que siempre se expande el nodo más prometedor, se ha implementado utilizando Dial's Bucket. En el algoritmo A*, la prioridad se basa en el coste total estimado ' $f(n) = g(n) + h(n)$ ', donde $g(n)$ es el coste real acumulado y $h(n)$ es el valor heurístico. En el algoritmo de Dijkstra, la heurística es siempre cero, por lo que la prioridad se basa únicamente en el coste acumulado $g(n)$.
- Lista Cerrada (Closed): almacena los nodos que ya han sido visitados y expandidos. Su propósito es evitar la re-expansión de nodos y prevenir ciclos, lo que mejora drásticamente la eficiencia. Se ha implementado utilizando una estructura de datos que permite búsquedas de alta velocidad, una tabla hash, para que la operación 'contains' sea, en promedio, de coste constante $O(1)$.

Ambos algoritmos implementados siguen el mismo flujo de trabajo. Esta implementación modular permite separar la lógica del algoritmo de las estructuras de datos y la representación del grafo, facilitando la comparación directa entre la búsqueda informada (A*) y la de fuerza bruta (Dijkstra) simplemente cambiando la forma en que se añaden y priorizan los nodos en la lista abierta: 'add_nh' para Dijkstra y 'add_h' para A*.

- Análisis de los resultados

Para analizar esta segunda parte, utilizamos la instancia USA-road-d.E proporcionada por *9th DIMACS Implementation Challenge - Shortest Paths* que contienen mapas de carreteras de la zona Este de Estados Unidos. Escogimos esta instancia para las pruebas ya que es un mapa suficientemente grande para probar distancias variadas sin que el programa tardara demasiado en inicializar el grafo. Esta instancia tiene 3598623 vértices y 8778114 arcos. Igual que antes, repetimos cada caso tres veces y obtenemos el tiempo medio.

- **Dijkstra**

par de vértices (s, t)	coste de la solución	tiempo medio (segundos)	expansiones	ratio nodos/seg
(1234567, 1234600)	51774	3.83	306	79.89
(1000, 1050)	586677	5.26	84947	16149.6
(900000, 2100000)	5205985	19.32	1324431	68552.33
(250000, 1500000)	4349713	38.13	2010992	52740.4

(5000, 3000000)	6386227	42.59	3075545	72212.84
(1, 3143888)	5318246	47.93	2757459	57530.96

- A*

par de vértices (s, t)	coste de la solución	tiempo medio (segundos)	expansiones	ratio nodos/seg
(1234567, 1234600)	53372	3.35	243	72.5
(1000, 1050)	597966	5.29	74645	14110.6
(900000, 2100000)	5242251	55.32	1148265	20756.8
(250000, 1500000)	4387468	97.46	1682875	17267.34
(5000, 3000000)	6460157	165.55	3001406	18129.9
(1, 3143888)	5438138	123.9	2665157	21510.5

La eficiencia de los algoritmos de búsqueda implementados depende en gran medida del tamaño que alcanzan las listas abierta y cerrada. Lista Cerrada: Su tamaño crece de forma monótonica durante la ejecución y es directamente proporcional al número de nodos expandidos. En el peor de los casos, podría llegar a contener una fracción significativa de todos los vértices del grafo. Lista Abierta: Su tamaño fluctúa, en cada iteración se extrae un nodo y se añaden sus sucesores. El tamaño máximo de esta lista representa el ancho máximo del frente de búsqueda.

Se ha elegido como algoritmo de búsqueda A* con heurística Haversine ya que es admisible y consistente, por lo que garantiza optimalidad y, por diseño, expande menos nodos que Dijkstra. La heurística Haversine redujo el número de nodos expandidos entre aproximadamente 2% y 21% según la consulta, cumpliendo el objetivo de expandir menos nodos que Dijkstra. No obstante, en la implementación actual en C++ el coste de calcular la heurística y el overhead de la gestión de la cola provocaron que A* fuera más lento en las consultas largas; en consultas cortas A* sí mostró mejora temporal.

En conclusión, A* intercambia un mayor coste computacional por nodo a cambio de una exploración mucho más inteligente y reducida del espacio de búsqueda.